

Phase 3 - Enhancements

(Complete Enhancements & UI/UX Features)

- **JWT role-based authentication**
 - **Pagination for employees & tasks**
 - **Drag-and-drop tasks with animations**
 - **Notifications for task updates**
 - **Attachments for tasks**
 - **Complex permission rules**
 - **Clean folder structure and naming conventions**
-

1 Objective

Phase 3 transforms your **working CRUD application** into a **real-world production-like task manager**, JIRA-style:

- Secure APIs using **JWT authentication**
 - Implement **role-based permissions**
 - Enable **pagination** for large datasets
 - Create **drag-and-drop task board with animations**
 - Show **notifications** for task changes
 - Support **task attachments**
 - Maintain **clean folder structure** for scalability
-

2 Roles & Permissions

Role	Permissions
Admin	CRUD on all employees & tasks, assign managers, view logs
Manager	Create tasks, assign tasks, update task status, view employees in own department
Employee	View own tasks, update status, upload attachments

Rules:

- Admin → full access
- Manager → can update tasks they created / assign tasks in their department
- Employee → can only update their own tasks' status

3 JWT Authentication

Login Endpoint:

POST /api/auth/login

Payload: { "email": "user@ust.com", "password": "Password123" }

Response: { "access_token": "...", "token_type": "bearer" }

Implementation Notes:

- Use **JWT middleware** to protect all CRUD endpoints
- Include `role` and `user_id` in JWT payload
- Backend checks **permissions based on role** for every action

4 Pagination

- **Employees:** `/api/employees?page=1&limit=20`
- **Tasks:** `/api/tasks?page=1&limit=10`

Backend returns:

```
{
  "data": [...],
```

```
"total": 120,  
"page": 1,  
"limit": 20  
}
```

- Frontend allows **page navigation** using buttons or infinite scroll

5 Drag-and-Drop Task Board (Frontend)

Columns: `TO_DO`, `IN_PROGRESS`, `REVIEW`, `DONE`

Implementation:

1. Use **React DnD** or **react-beautiful-dnd**
2. **Draggable task cards** with smooth animations
3. **Column highlights** when dragging over

Axios PUT API call on drop:

```
const handleDragEnd = (result) => {  
  if (!result.destination) return;  
  const { draggableId, destination } = result;  
  const newStatus = destination.droppableId;  
  
  axios.put(`/api/tasks/${draggableId}`, { status: newStatus })  
    .then(() => fetchTasks()); // refresh tasks  
};
```

Animations & UI polish (TailwindCSS):

```
<div className="task-card bg-white p-4 rounded shadow-md hover:shadow  
-xl transition-shadow duration-300">  
  Update client database  
</div>
```

- Color-coded columns:

- TO_DO → Blue
- IN_PROGRESS → Yellow
- REVIEW → Orange
- DONE → Green

6 Notifications

- Show **real-time toast notifications** using **react-toastify**:

```
toast.success("Task 'Update DB' moved to IN_PROGRESS!");
```

- Notify users when:
 - Task assignedask status updated
 - Task completed

7 Task Attachments

- Upload files via **FormData**:

```
POST /api/tasks/:id/attachments
```

```
FormData: file
```

```
Response: { "message": "Attachment uploaded successfully", "url": "/files/file  
name.pdf" }
```

- Show attachments inside **task card** as icons or thumbnails
- Hover effect → tooltip with filename
- Upload animation → progress bar while uploading

8 Backend Folder Structure Updates

```
ust-employee-backend/  
├─ app/
```

```

|   ├── routers/
|   |   ├── auth_router.py      # JWT login
|   |   ├── employee_router.py  # CRUD + pagination
|   |   └── task_router.py      # CRUD + pagination + attachments
|   ├── middleware/
|   |   ├── auth_middleware.py  # JWT & role check
|   |   └── logging_middleware.py # MongoDB logging
|   ├── services/
|   |   ├── employee_service.py
|   |   ├── task_service.py
|   |   └── notification_service.py
|   └── utils/
|       └── file_upload.py      # attachment helper
├── uploads/                   # store uploaded files
├── requirements.txt
└── .env

```

9 Frontend Folder Structure (Phase 3)

```

ust-employee-frontend/
├── src/
|   ├── components/
|   |   ├── TaskBoard.jsx      # Drag-and-drop task board
|   |   ├── TaskCard.jsx      # Single task card with animations
|   |   ├── EmployeeForm.jsx
|   |   └── TaskForm.jsx
|   ├── pages/
|   |   ├── Dashboard.jsx
|   |   └── Login.jsx
|   ├── services/
|   |   ├── employeeService.js
|   |   └── taskService.js
|   └── utils/
|       └── notifications.js    # toast helper

```

10 Phase 3 Deliverables

1. **JWT authentication and role-based access**
 2. **Pagination** for employees and tasks
 3. **Drag-and-drop task board** with smooth animations
 4. **Notifications** for task events
 5. **Task attachments** upload & preview
 6. **Color-coded, eye-catching UI** (Tailwind)
 7. **Backend folder structure with logging, services, and middleware**
-